

Student

I've built facts, dimensions, and SCD. Now I keep hearing **star schema** and **snowflake schema**. What's the difference, and which do I pick?

ChatGPT • AI Assistant

These are the two dominant **schema patterns** for organizing analytical data. Both use facts and dimensions from Chapter 2 - the difference is how the **dimensions are normalized**. This chapter teaches:

1. What a **schema pattern** means in a warehouse context.
2. The **star schema** - fully denormalized, Kimball's default.
3. The **snowflake schema** - partially normalized dimensions.
4. When to pick **star vs snowflake** - the trade-off matrix.
5. **Galaxy (fact constellation)** schemas - multiple facts, shared dims.
6. **Performance considerations** - indexing, partitioning, clustering, aggregates.
7. **Common modeling pitfalls** you must avoid.
8. A **hands-on conversion** - take a star, snowflake it, then unsnowflake it back.

CONCEPT CARD

A **star schema** has a central fact table joined to fully denormalized dimensions - the dimension tables carry every attribute of their entity flatly. A **snowflake schema** takes those same dimensions and breaks each into normalized sub-tables (e.g., `dim_product` → `dim_product` + `dim_category` + `dim_brand`).

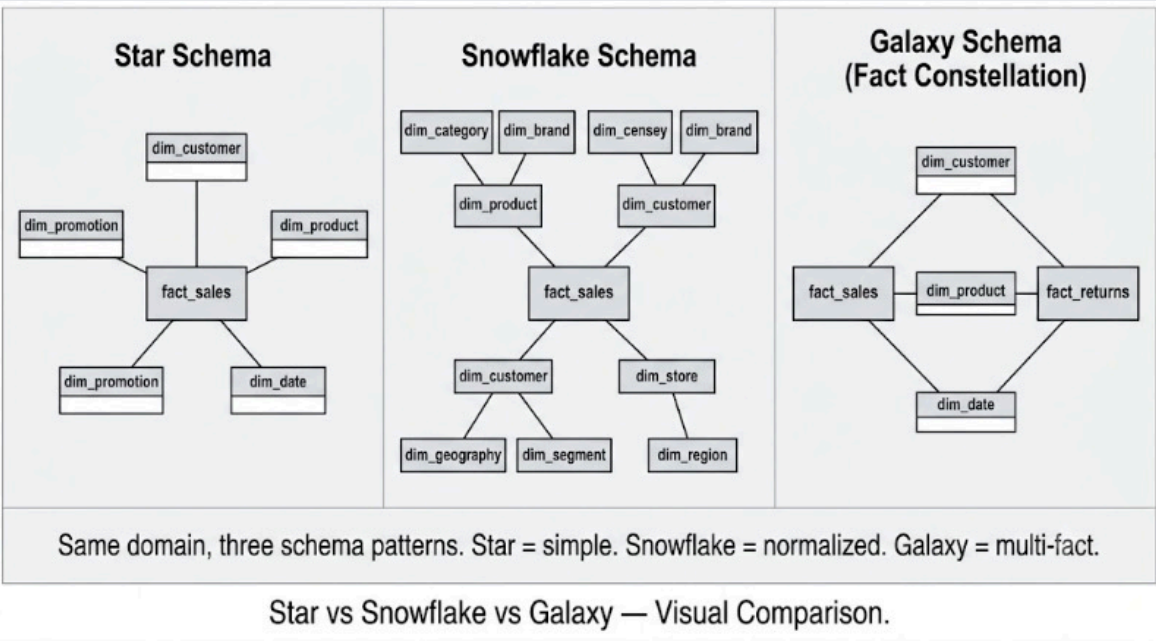
Both are valid; star is simpler and faster; snowflake is more normalized and modular.

SUMMARY CARD

By the end of this chapter you'll be able to:

- Recognize a star vs a snowflake at a glance.
- Choose between them based on query patterns, storage, and governance needs.
- Design a **galaxy schema** for a multi-process warehouse.
- Diagnose and fix common modeling pitfalls.
- Convert between star and snowflake on demand.

[AI Prompt] Warm-up (paste into any AI chatbot): "Show me the same e-commerce sales analytics domain modeled two ways: (1) as a star schema with denormalized dim_product, and (2) as a snowflake with dim_product broken into product/category/brand tables. Include a revenue-by-category query for each and compare join complexity."



Star vs Snowflake vs Galaxy — Visual Comparison

Star vs Snowflake — Head-to-Head Trade-offs

Aspect	Star	Snowflake
Dimension shape	Fully denormalized (wide)	Normalized into sub-tables
Number of tables	Fewer (1 fact + N dims)	More (dims split further)
Storage	Higher (duplicated attributes)	Lower (no duplication)
Query joins	Few (fact + 1-2 dims)	Many (fact + dim + sub-dim)
Query speed	Fast	Slower on large data
Query readability	Simple SQL	Complex SQL
Maintenance	Update dim in many rows	Update central sub-table once

Aspect	Star	Snowflake
BI tool support	Excellent (native)	Requires more configuration
Best for	Analytics teams, dashboards	Governed hierarchies, huge dims

sql – Same domain, both patterns – the essential difference

```
-- =====
-- STAR: dim_product carries category and brand flatly
-- =====

CREATE TABLE dim_product (
  product_key    BIGINT      PRIMARY KEY,
  sku            VARCHAR(50) NOT NULL,
  product_name   VARCHAR(200),
  category_code  VARCHAR(20),
  category_name  VARCHAR(100),
  brand_code     VARCHAR(20),
  brand_name     VARCHAR(100),
  unit_cost      DECIMAL(12, 2)
);

-- Revenue by category: ONE join
SELECT p.category_name, SUM(s.net_amount) AS revenue
FROM   fact_sales s
JOIN   dim_product p ON p.product_key = s.product_key
GROUP BY p.category_name;

-- =====
-- SNOWFLAKE: category and brand normalized into sub-tables
-- =====

CREATE TABLE dim_category (
  category_key    BIGINT      PRIMARY KEY,
  category_code   VARCHAR(20) NOT NULL,
  category_name   VARCHAR(100)
);

CREATE TABLE dim_brand (
  brand_key       BIGINT      PRIMARY KEY,
  brand_code      VARCHAR(20) NOT NULL,
  brand_name      VARCHAR(100)
);
```

```
CREATE TABLE dim_product (  
    product_key    BIGINT        PRIMARY KEY,  
    sku            VARCHAR(50)   NOT NULL,  
    product_name   VARCHAR(200),  
    category_key   BIGINT        REFERENCES dim_category(category_key),  
    brand_key      BIGINT        REFERENCES dim_brand(brand_key),  
    unit_cost      DECIMAL(12, 2)  
);  
  
-- Revenue by category: THREE joins  
SELECT c.category_name, SUM(s.net_amount) AS revenue  
FROM   fact_sales    s  
JOIN   dim_product   p ON p.product_key = s.product_key  
JOIN   dim_category  c ON c.category_key = p.category_key  
GROUP BY c.category_name;
```

WHAT IS A SCHEMA IN A DATA WAREHOUSE?

Student

In warehousing, when we say **schema**, do we mean the same thing as in OLTP?

ChatGPT • AI Assistant

Similar term, different focus. In OLTP, a schema is usually a **3NF entity-relationship diagram** designed for updates.

In a warehouse, a **schema** is the **overall shape** of how facts and dimensions are organized - the pattern of connections, not just the tables themselves. Three main patterns dominate: **star**, **snowflake**, and **galaxy** (a.k.a. fact constellation).

CONCEPT CARD

The three warehouse schema patterns:

1. **Star schema** - one central fact + denormalized dimensions radiating outward.
2. **Snowflake schema** - a star, but with dimensions normalized into sub-tables (extra branches).

3. **Galaxy / fact constellation** - multiple fact tables sharing conformed dimensions.

Student

Is a warehouse schema always one of these three?

ChatGPT • AI Assistant

EXAMPLE CARD

In practice, yes:

1. **Small warehouses** (single business process) → usually **star**.
2. **Warehouses with hierarchies managed independently** (e.g., product taxonomy governed centrally) → **snowflake**.
3. **Enterprise warehouses** (many business processes, shared dims) → **galaxy** - which is really multiple stars sharing dimensions.
4. Real teams often **mix**: a mostly-star warehouse with one snowflake dim for a huge product hierarchy.

🔗 ANALOGY

Schema patterns are like **floor plans** for a warehouse:

- **Star** = single big room with satellite closets around it - easy to navigate, everything's close.
- **Snowflake** = same big room, but each closet has its own sub-closets - more organized, more doors to open.
- **Galaxy** = multiple big rooms sharing closets - necessary once your team is doing many business processes.

⚠️ WARNING

Don't confuse OLTP 3NF with snowflake. Snowflake dimensions **partially** normalize; they're still analytics-optimized. A pure 3NF schema would be a **completely** different animal.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Thinking there's one "correct" schema. There isn't - the right one depends on your query patterns, team size, and governance needs.

Correct Way: Learn all three, then pick based on trade-offs. Star is the default; the others solve specific problems.

EXAM TIP CARD

Interview classic: *"What are the three warehouse schema patterns?"* Winning answer: *"Star (fact + denormalized dims), snowflake (star with normalized dim sub-tables), and galaxy or fact constellation (multiple facts sharing conformed dims). Star is Kimball's default; the others solve specific normalization and multi-process needs."*

🔥 EXERCISE

For any warehouse you know of, sketch its high-level shape on paper. Is it star, snowflake, galaxy - or a mix? Which parts feel over- or under-normalized?

[AI Prompt] Paste this into any AI chatbot: *"Explain star, snowflake, and galaxy warehouse schemas using an e-commerce analogy for each. For each, describe when it's the right choice."*

🔗 PRACTICE QUIZ

Which schema pattern uses multiple fact tables sharing the same dimensions?

Answer: **Galaxy schema** (also called a **fact constellation**).

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

THE STAR SCHEMA

ChatGPT • AI Assistant

```
sql – Retail star schema – everything the analytics team needs
```

```
-- =====  
-- Star schema for retail sales  
-- Every dim is FLAT – all attributes in one row  
-- =====
```

```
CREATE TABLE dim_date (  
    date_key          INT          PRIMARY KEY,  
    full_date         DATE         NOT NULL,  
    day_name          VARCHAR(10),  
    month_name        VARCHAR(10),
```

```

    quarter_name    VARCHAR(10),
    year            INT,
    is_weekend      BOOLEAN,
    is_holiday      BOOLEAN
);

CREATE TABLE dim_customer (
    customer_key    BIGINT          PRIMARY KEY,
    customer_id     VARCHAR(50)    NOT NULL,
    full_name       VARCHAR(200),
    city            VARCHAR(100),
    region          VARCHAR(50),    -- FLAT: not a separate dim
    country         VARCHAR(80),    -- FLAT
    segment         VARCHAR(50)     -- FLAT
);

CREATE TABLE dim_product (
    product_key     BIGINT          PRIMARY KEY,
    sku             VARCHAR(50)    NOT NULL,
    product_name    VARCHAR(200),
    category        VARCHAR(100),   -- FLAT: not a separate dim
    sub_category    VARCHAR(100),   -- FLAT
    brand           VARCHAR(100),   -- FLAT
    supplier_name   VARCHAR(200),   -- FLAT
    unit_cost       DECIMAL(12, 2)
);

CREATE TABLE dim_store (
    store_key       BIGINT          PRIMARY KEY,
    store_id        VARCHAR(20)    NOT NULL,
    store_city      VARCHAR(100),
    store_region    VARCHAR(50),    -- FLAT
    store_format    VARCHAR(20)
);

CREATE TABLE fact_sales (
    sales_key       BIGINT          PRIMARY KEY,
    date_key        INT            REFERENCES dim_date(date_key),
    customer_key    BIGINT          REFERENCES dim_customer(customer_key),
    product_key     BIGINT          REFERENCES dim_product(product_key),
    store_key       BIGINT          REFERENCES dim_store(store_key),
    order_id        VARCHAR(50),    -- degenerate dim
    quantity        INT            NOT NULL,
    unit_price      DECIMAL(12, 2),
    net_amount      DECIMAL(12, 2) NOT NULL,
    discount_amt    DECIMAL(12, 2) DEFAULT 0

```

```
);  
  
-- Sample report: revenue by category and region  
SELECT p.category, c.region, SUM(s.net_amount) AS revenue  
FROM   fact_sales   s  
JOIN    dim_product  p ON p.product_key = s.product_key  
JOIN    dim_customer c ON c.customer_key = s.customer_key  
GROUP  BY p.category, c.region;
```

Student

Show me what makes a **star schema** a star.

ChatGPT • AI Assistant

A **star schema** has exactly one central **fact table** surrounded by **fully denormalized dimension tables**. Every dimension attribute - even hierarchical ones like category, brand, region - sits **flat** in its own dimension row.

Draw the fact in the middle, dimensions around it, and connect them with FK lines: you literally get a star.

CONCEPT CARD

Star schema hallmarks:

1. **One fact table** at the center, containing FKs + measures.
2. **Dimensions are wide and denormalized** - even nested attributes are flattened.
3. **All joins are one hop** - fact ↔ dim, never dim ↔ dim.
4. **Query SQL is short** - most reports are two-way joins.
5. **Kimball's preferred pattern** - favors query simplicity over storage.

Student

Give me a concrete star schema for retail sales.

ChatGPT • AI Assistant

EXAMPLE CARD

Reference star for retail (which you've already been building in previous chapters):

1. ``fact_sales`` - one row per line item. FKs to `dimdate`, `dimcustomer`, `dimproduct`, `dimstore`, `dimpromotion`. Measures: *quantity*, *netamount*, *discountamount*, *taxamount*.
2. ``dim_date`` - one row per day with 20+ calendar attributes.
3. ``dim_customer`` - customer id, name, email, city, region, segment (all flat).
4. ``dim_product`` - SKU, product name, category, sub_category, brand, supplier (all flat).
5. ``dim_store`` - store id, city, region, format (all flat).
6. ``dim_promotion`` - promo id, name, discount type, campaign (all flat).

Every report is: `fact_sales JOIN 1-3 dims` . That's it.

🔗 ANALOGY

A star schema is like a **wall clock**. The center is the fact - the source of readings. The dimensions are the numbered marks around it - each provides one dimension of context, all equally close, all one glance away.

⚠️ WARNING

The star sacrifices some storage - the same category name is repeated on every product row. But storage is cheap and query simplicity is priceless. Modern column-store warehouses (Snowflake, BigQuery, Redshift) barely feel the redundancy.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Building a "star" but leaving hierarchies in separate tables. That's actually a snowflake in disguise - you lost the simplicity benefit.

Correct Way: Fully denormalize each dim. Category, subcategory, brand all live in ``dimproduct``.

EXAM TIP CARD

Interview classic: *"Why is a star schema faster than a snowflake?"* Winning answer: *"Fewer joins. In a star, every attribute a report needs is one hop away - the fact directly joins the dimension. Fewer joins mean less optimizer work and faster execution on huge fact tables."*

🔥 EXERCISE

Sketch a star schema for a book publisher: `fact_book_sales` at the center; dimensions for date, book, author, publisher, retailer, promotion. List every attribute you'd flatten INTO the book and author dims.

[AI Prompt] Paste this into any AI chatbot: *"Design a star schema for streaming service analytics with fact_stream_events at the center and dimensions for date,*

subscriber, title, device, and country. List 15-20 attributes per dimension."

THE SNOWFLAKE SCHEMA

ChatGPT • AI Assistant

sql – Snowflake schema – dim_product broken into sub-tables

```
-- =====  
-- Snowflake: dim_product now has FKs into sub-dims  
-- =====  
  
CREATE TABLE dim_department (  
    department_key    BIGINT        PRIMARY KEY,  
    department_code   VARCHAR(20)   NOT NULL,  
    department_name    VARCHAR(100)  
);  
  
CREATE TABLE dim_category (  
    category_key      BIGINT        PRIMARY KEY,  
    category_code     VARCHAR(20)   NOT NULL,  
    category_name     VARCHAR(100),  
    department_key    BIGINT  
                    REFERENCES dim_department(department_key)  
);  
  
CREATE TABLE dim_brand (  
    brand_key         BIGINT        PRIMARY KEY,  
    brand_code        VARCHAR(20)   NOT NULL,  
    brand_name        VARCHAR(100),  
    supplier_name     VARCHAR(200)  
);  
  
CREATE TABLE dim_product (  
    product_key       BIGINT        PRIMARY KEY,  
    sku               VARCHAR(50)   NOT NULL,  
    product_name      VARCHAR(200),  
    category_key      BIGINT        REFERENCES dim_category(category_key),  
    brand_key         BIGINT        REFERENCES dim_brand(brand_key),  
    unit_cost         DECIMAL(12, 2)  
);
```

```
-- Same fact table as before
CREATE TABLE fact_sales (
    sales_key      BIGINT      PRIMARY KEY,
    date_key       INT         REFERENCES dim_date(date_key),
    customer_key   BIGINT      REFERENCES dim_customer(customer_key),
    product_key    BIGINT      REFERENCES dim_product(product_key),
    store_key      BIGINT      REFERENCES dim_store(store_key),
    quantity       INT,
    net_amount     DECIMAL(12, 2)
);

-- Revenue by department: FOUR joins instead of TWO
SELECT dep.department_name,
       SUM(s.net_amount) AS revenue
FROM   fact_sales      s
JOIN   dim_product     p  ON p.product_key   = s.product_key
JOIN   dim_category    c  ON c.category_key  = p.category_key
JOIN   dim_department  dep ON dep.department_key = c.department_key
GROUP BY dep.department_name;
```

Student

What makes a schema a **snowflake**?

ChatGPT • AI Assistant

A **snowflake schema** is a star, but with dimensions **normalized into sub-tables**.

Instead of **dim_product** holding category and brand attributes flatly, it holds **category_key** and **brand_key** FKs pointing to separate **dim_category** and **dim_brand** tables.

When drawn, dimensions have branches on branches - like a snowflake.

CONCEPT CARD

Snowflake schema hallmarks:

1. **Fact still central**, dimensions still around it.
2. Each dimension is **broken into normalized sub-tables** - often based on hierarchies (category → sub-category → department).
3. **More tables**, more joins per report.
4. **Less storage duplication** - each attribute stored once.

5. **Easier centralized governance** - one authoritative `dim_category` used everywhere.

Student

Why would anyone snowflake?

ChatGPT • AI Assistant

EXAMPLE CARD

Legitimate reasons to snowflake:

1. **Very large dimensions** - a `dim_product` with 50 million rows and heavy hierarchies benefits from splitting.
2. **Hierarchies managed independently** - the product-taxonomy team owns categories; the brand-management team owns brands. Separate tables map to separate teams.
3. **Frequent hierarchy updates** - rebranding? Updating one central `dim_brand` beats updating millions of product rows.
4. **Compliance / audit** - reference data (currency codes, country codes) sometimes must live in its own table for governance.
5. **Data quality gates** - a validated sub-dim can be the single source of truth for its entity.

🔗 ANALOGY

A snowflake schema is like a **branching family tree**. Instead of every person's row holding their entire ancestry flatly, you have separate person / parent / grandparent tables. More precise, but more clicks to trace a lineage.

⚠️ WARNING

Every extra join costs performance on huge fact tables. If your product dim has 10K rows, snowflaking it saves negligible storage and doubles query complexity - don't do it.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Snowflaking every dimension "because it's cleaner." Reports become 6-way joins; BI tools struggle; analysts complain.

Correct Way: Snowflake **selectively** - only where a dim is huge, or where governance genuinely benefits from separation.

EXAM TIP CARD

Interview classic: *"When would you choose a snowflake over a star?"* Winning answer: *"When a dimension is very large, when hierarchies are governed by separate teams, or when compliance requires reference data to live in its own table. For most warehouses, star is the default; snowflake is a targeted exception."*

🔥 EXERCISE

Take the `dim_product` from Section 4-2. Snowflake it: pull category, sub_category, brand, and supplier into their own dim tables. Sketch the new schema - count the joins for the same revenue-by-category report.

[AI Prompt] Paste this into any AI chatbot: *"Take a star schema with dim_product (flat) and convert it into a snowflake by extracting category, brand, and supplier into their own tables. Show DDL and the equivalent revenue-by-category query for both - compare join count and readability."*

STAR VS SNOWFLAKE — WHEN TO USE WHICH

Student

Give me a **decision framework** for star vs snowflake.

ChatGPT • AI Assistant

Default to star. Snowflake is a targeted exception justified by specific conditions - large dims, governance needs, or independently-managed hierarchies. **When in doubt, star.**

CONCEPT CARD

Choose **star** when:

1. Analytics team is small or business-user-driven.
2. Dimensions are moderate in size (thousands to a few million rows).
3. BI tools (Tableau, Power BI, Looker) will query directly.
4. Query simplicity and speed matter more than storage.
5. You don't have separate teams governing hierarchies.

Choose **snowflake** when:

1. A dimension is genuinely huge (10M+ rows with heavy hierarchies).
2. A separate team owns the hierarchy independently (product taxonomy, geographic hierarchy).
3. Reference data must be centralized for compliance.
4. Storage cost dominates the query cost calculation.
5. Frequent hierarchy updates make Type 2 on the flat dim impractical.

Student

Can I mix?

ChatGPT • AI Assistant

EXAMPLE CARD

Absolutely - hybrid is normal:

1. Most dims stay star (denormalized).
2. One or two dims get snowflaked selectively (e.g., a giant product hierarchy).
3. The rest of the warehouse doesn't care.
4. This is often called a **starflake** - a mostly-star schema with a few snowflake branches.

🔗 ANALOGY

Choosing between star and snowflake is like choosing between a **studio apartment** and a **two-bedroom** - depends on how much stuff you have and how many people are using it. Most singles are fine in a studio; families need the extra room.

⚠️ WARNING

Once you snowflake a dim, BI-tool configuration gets more complex. Users must learn the extra joins, semantic-layer definitions grow, and cross-team coordination increases. Snowflake only when the payoff is worth it.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Debating star vs snowflake as a religious choice. Both are valid; pick per dimension based on real trade-offs.

Correct Way: Start star. Snowflake specific dims only when you have a concrete reason (size, governance, hierarchy independence).

EXAM TIP CARD

Interview classic: *"Star or snowflake - which is better?"* Winning answer: *"Neither is universally better. Star is the default for simplicity and query speed.*

Snowflake is a targeted choice for large dims, governance separation, or independently-managed hierarchies. Most real warehouses use a hybrid - 'starflake'."

🔥 EXERCISE

For each of these dims, decide star or snowflake and justify: (1) 5,000 products with 3-level hierarchy, (2) 10 million products across 50 categories, (3) `dim_date` with 20 years of dates, (4) `dim_geography` with 250 countries + 5000 cities.

[AI Prompt] Paste this into any AI chatbot: *"For a global e-commerce warehouse, tell me which dimensions I should snowflake and which I should keep flat. Justify each based on typical sizes and governance patterns."*

GALAXY (FACT CONSTELLATION) SCHEMAS

ChatGPT • AI Assistant

sql – Cross-fact analysis on a galaxy schema

```
-- =====
-- Galaxy: fact_sales + fact_returns share dim_customer,
-- dim_product, and dim_date. Join across facts via dims.
-- =====

-- Question: Which customer segments have the highest
-- return-to-purchase ratio in Q3 2026?

WITH sales_by_segment AS (
    SELECT c.segment,
           SUM(s.net_amount) AS gross_revenue
    FROM   fact_sales s
    JOIN   dim_customer c ON c.customer_key = s.customer_key
    JOIN   dim_date d ON d.date_key = s.date_key
    WHERE  d.year = 2026 AND d.quarter_name = 'Q3 2026'
    GROUP BY c.segment
),
returns_by_segment AS (
    SELECT c.segment,
```

```

        SUM(r.refund_amount) AS returns
    FROM    fact_returns r
    JOIN    dim_customer c ON c.customer_key = r.customer_key
    JOIN    dim_date      d ON d.date_key      = r.return_date_key
    WHERE   d.year = 2026 AND d.quarter_name = 'Q3 2026'
    GROUP   BY c.segment
)
SELECT s.segment,
       s.gross_revenue,
       COALESCE(r.returns, 0) AS returns,
       ROUND(COALESCE(r.returns, 0) * 100.0 / s.gross_revenue,
             2) AS return_rate_pct
FROM    sales_by_segment s
LEFT JOIN returns_by_segment r ON r.segment = s.segment
ORDER   BY return_rate_pct DESC;

-- The two facts join via the SAME dim_customer (conformed).
-- That's the whole point of the galaxy pattern.

```

Student

What's a **galaxy schema**?

ChatGPT • AI Assistant

A **galaxy schema** (also called a **fact constellation**) is a warehouse with **multiple fact tables sharing conformed dimensions**. It's what happens naturally when you build stars for multiple business processes (sales, returns, inventory, support) and reuse the same `dim_date`, `dim_customer`, `dim_product` across all of them.

CONCEPT CARD

Why galaxies matter:

1. **Conformed dimensions** (Chapter 2) enable cross-fact analysis.
2. Each business process gets its own **fact table** - appropriate grain, appropriate measures.
3. **Shared dims** guarantee consistency - one "active customer count" definition.
4. Adding a new business process just adds a new fact; existing reports are unaffected.
5. **This is what real enterprise warehouses look like.**

Give me a concrete galaxy example.

ChatGPT • AI Assistant

EXAMPLE CARD

An enterprise retail warehouse might have:

1. ``fact_sales`` - one row per line item (transaction grain).
2. ``fact_returns`` - one row per returned item (transaction grain).
3. ``fact_inventory_daily`` - one row per SKU per store per day (periodic snapshot).
4. ``fact_support_ticket`` - one row per ticket status change (transaction grain).
5. Shared conformed dims: `dim_date` , `dim_customer` , `dim_product` , `dim_store` , `dim_employee` .

Each fact is its own star. Together they form a galaxy - and you can join across facts via shared dims (e.g., "customers with high support tickets AND high returns").

🔗 ANALOGY

A galaxy schema is like a **shopping mall with shared parking**. Each store (fact table) has its own layout and inventory (grain and measures). But everyone shares the same parking (conformed dims). Move between stores freely - the parking connects them.

⚠️ WARNING

Galaxy schemas require **disciplined dimension management**. If one team creates a private `dim_customer_v2` , the whole galaxy fragments and cross-fact analysis breaks. Assign a data-platform team to own conformed dims.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Each business process building its own separate `dim_customer`. Six months later, no one can produce a single trusted customer count across processes.

Correct Way: One centrally-maintained `dim_customer` , referenced by every fact table via the surrogate `customer_key` .

EXAM TIP CARD

Interview classic: "What is a galaxy schema?" Winning answer: "Multiple fact tables sharing conformed dimensions - the natural output of the Kimball bus architecture. Enables cross-process analysis and a single source of truth per dimension."

EXERCISE

Sketch a galaxy schema for a food-delivery company: facts for orders, deliveries, refunds, customer-support-tickets. List the conformed dims each fact needs. That's the bus matrix from Chapter 2 in schema form.

[AI Prompt] Paste this into any AI chatbot: *"Design a galaxy schema for a ride-sharing app with facts for trips, cancellations, driver payments, and complaints. Show conformed dimensions and one cross-fact query joining two facts via a shared dim."*

PERFORMANCE CONSIDERATIONS

ChatGPT • AI Assistant

sql – Performance patterns – indexes, partitions, clustering, aggregates

```
-- =====
-- 1) FK indexes on the fact table
-- =====

CREATE INDEX idx_fs_date      ON fact_sales(date_key);
CREATE INDEX idx_fs_customer ON fact_sales(customer_key);
CREATE INDEX idx_fs_product  ON fact_sales(product_key);
CREATE INDEX idx_fs_store    ON fact_sales(store_key);

-- =====
-- 2) Partition the fact table by month (PostgreSQL style)
-- =====

CREATE TABLE fact_sales (
    sales_key      BIGINT,
    date_key       INT NOT NULL,
    customer_key   BIGINT,
    product_key    BIGINT,
    net_amount     DECIMAL(12, 2)
)
PARTITION BY RANGE (date_key);

CREATE TABLE fact_sales_2026_q1
    PARTITION OF fact_sales
    FOR VALUES FROM (20260101) TO (20260401);

CREATE TABLE fact_sales_2026_q2
```

```

PARTITION OF fact_sales
FOR VALUES FROM (20260401) TO (20260701);

-- =====
-- 3) Clustering (Snowflake) – physical layout by common filter
-- =====
-- ALTER TABLE fact_sales CLUSTER BY (date_key, store_key);

-- =====
-- 4) Pre-aggregated table – refreshed nightly by ETL
-- =====

CREATE TABLE agg_daily_revenue_by_category (
  date_key      INT          NOT NULL,
  category      VARCHAR(100) NOT NULL,
  revenue       DECIMAL(14, 2),
  units_sold    INT,
  unique_customers INT,
  PRIMARY KEY (date_key, category)
);

-- Reports hit the aggregate first, only fall back to base fact
-- for granular questions.
SELECT date_key, category, revenue
FROM   agg_daily_revenue_by_category
WHERE  date_key BETWEEN 20260701 AND 20260930
ORDER  BY revenue DESC;

```

Student

What actually makes star / snowflake queries **fast** or **slow**?

ChatGPT • AI Assistant

Performance in a warehouse comes from four levers: **indexing**, **partitioning**, **clustering / sort keys**, and **pre-aggregated tables**. Modern column-store warehouses (Snowflake, BigQuery, Redshift) handle much of this automatically - but understanding the levers lets you diagnose and tune when needed.

CONCEPT CARD

The four performance levers:

1. **Index FKs on fact tables** - every dimension FK gets an index. Speeds up joins massively.
2. **Partition facts by date** - physically split the fact table by `date_key` month or day. Reports filter to relevant partitions only.
3. **Cluster / sort by common filters** - order the fact rows by the columns most reports filter on (usually date + geography).
4. **Materialized aggregate tables** - pre-compute common rollups (daily revenue by category) into `agg_ ...` tables refreshed nightly.

Student

What does a column store buy me?

ChatGPT • AI Assistant

EXAMPLE CARD

Column-store warehouses (Snowflake, BigQuery, Redshift, ClickHouse) rewrite the physics:

1. Only relevant columns are read from disk - a query on `net_amount` doesn't scan the other 20 columns.
2. Columns are compressed heavily - 5-10× reduction is common.
3. Vectorized execution processes millions of rows per second per CPU.
4. Denormalized wide dimensions cost almost nothing extra - column stores ignore unused columns.
5. Star schemas literally get faster in column stores; snowflaking hurts more.

🔗 ANALOGY

A column store is like reading a **library card catalog by column** (author, title, year) rather than reading each full book. If your query only needs authors and years, you never touch the rest.

⚠️ WARNING

Avoid `SELECT *` on wide dimensions in column stores. The engine dutifully reads every column and defeats the compression benefit. Always select only the columns your report needs.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Pre-aggregating everything you *might* need. Aggregates become stale, storage bloats, and users trust the raw fact more.

Correct Way: Aggregate only the top-N most-used reports (daily revenue, monthly active users). Let ad-hoc queries hit the base fact.

EXAM TIP CARD

Interview classic: *"How do you speed up a slow star-schema report?"* Winning answer: *"First, check partition pruning on date_key. Then check FK indexes on the fact."*

Then examine query shape - often a poorly-written filter defeats the optimizer. Materialized aggregates are the last resort."

🔥 EXERCISE

For any warehouse you can access, run **EXPLAIN** on a common report. Look for: partition pruning, index usage, and full table scans. That's your performance map.

[AI Prompt] Paste this into any AI chatbot: *"Give me 5 concrete techniques to speed up star-schema queries in Snowflake or BigQuery. For each, explain when to use it and how to verify it's helping."*

COMMON MODELING PITFALLS

Student

What are the **top mistakes** I should avoid?

ChatGPT • AI Assistant

Six pitfalls trip up nearly every beginner. Learn to spot them in your own designs - and in code reviews.

CONCEPT CARD

The six deadly modeling sins:

1. **Mixed grain** - some rows are line items, others are totals. Sums double-count.
2. **Missing surrogate keys** - natural keys as PKs kill SCD Type 2 and hurt joins.
3. **Fact-to-fact joins** - trying to join two facts directly. Always join facts via shared conformed dimensions.
4. **Accidental snowflaking** - building a "star" but leaving hierarchies in separate tables.

5. **Overly wide dims** - 500 columns because "maybe we'll need it." Bloat + confusion.
6. **Undated attributes on facts** - descriptive text on the fact instead of the dim. History breaks and updates are painful.

Student

Can you show one bad and one good example?

ChatGPT • AI Assistant

EXAMPLE CARD

Mixed-grain nightmare:

1. `fact_sales` has some rows at line-item grain (quantity=1, unit_price=100, amount=100) AND some rows at order-total grain (order 12345, amount=500).
2. `SUM(amount)` counts the same money twice.
3. Fix: separate into `fact_sales_line` and `fact_sales_order`.

🔗 ANALOGY

These pitfalls are the **potholes on your warehouse road**. You'll drive over some of them once - the goal is to recognize them fast enough that you patch rather than crash.

⚠️ WARNING

Fact-to-fact joins deserve special mention: they almost always indicate mixed-grain or missing conformed dims. If you find yourself writing `fact_sales JOIN fact_shipments`, stop and check whether you should join via `dim_customer` or `dim_order` instead.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Adding a `customer_name` column directly to `fact_sales` "to avoid joins." When the name changes, past facts show the wrong name; SCD becomes impossible.

Correct Way: Only FKs and measures on facts. Descriptive text lives in dims and is versioned via SCD Type 2.

EXAM TIP CARD

Interview favorite: *"Tell me about a data-modeling mistake you've fixed."* Winning answer template: *"Discovered mixed grain in fact_X - SUM(measure) was double-*

counting. Fixed by declaring one strict grain, splitting into two facts. Reports came back to reality overnight."

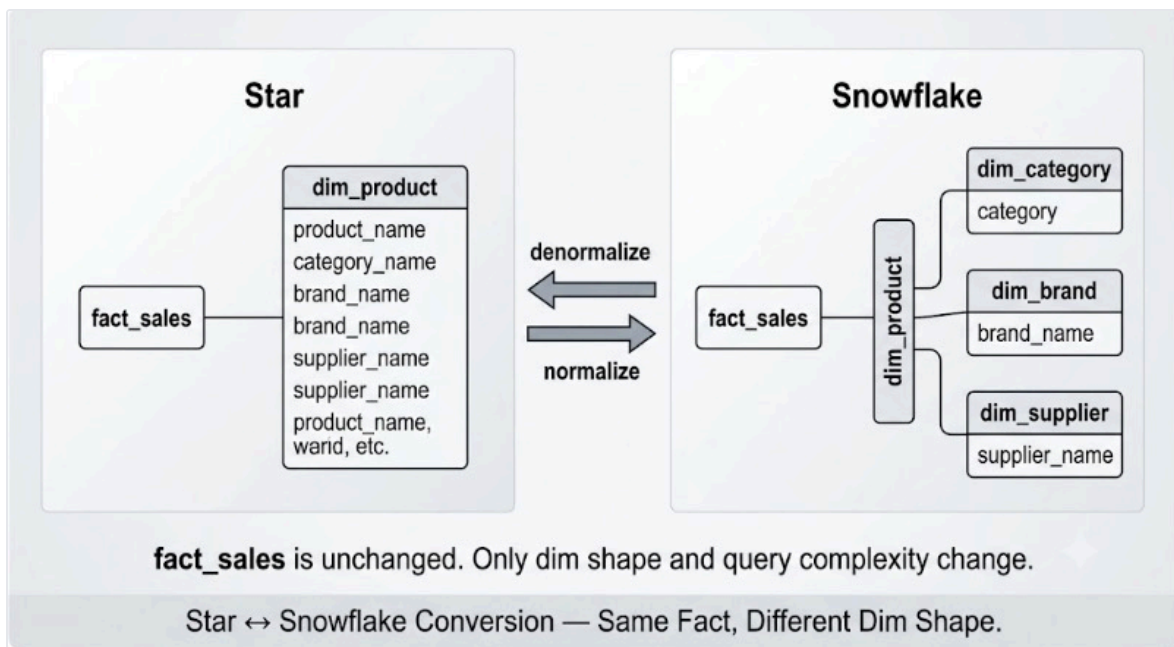
🔥 EXERCISE

Audit any star schema you have access to. For each fact table, verify: (1) grain is documented and consistent, (2) all FKs point to surrogate keys, (3) no descriptive text lives on the fact, (4) no accidental sub-dims.

[AI Prompt] Paste this into any AI chatbot: "Show me 6 common data-modeling anti-patterns in dimensional design. For each, describe the symptom, the risk, and the fix."

HANDS-ON — CONVERT A STAR TO SNOWFLAKE AND BACK

ChatGPT • AI Assistant



Star ↔ Snowflake Conversion — Same Fact, Different Dim Shape

sql — Full star ↔ snowflake conversion — DDL and data movement

```
-- =====
-- STARTING POINT: STAR SCHEMA (flat dim_product)
-- =====

CREATE TABLE dim_product_star (
  product_key  BIGINT      PRIMARY KEY,
  sku          VARCHAR(50),
```

```

product_name    VARCHAR(200),
category_code   VARCHAR(20),
category_name   VARCHAR(100),
brand_code      VARCHAR(20),
brand_name      VARCHAR(100),
supplier_name   VARCHAR(200),
unit_cost       DECIMAL(12, 2)
);

-- Sample query: 2 joins
-- SELECT p.category_name, SUM(s.net_amount)
-- FROM   fact_sales s JOIN dim_product_star p
--       ON p.product_key = s.product_key
-- GROUP BY p.category_name;

-- =====
-- CONVERT STAR → SNOWFLAKE
-- =====

-- Step 1: create the sub-dim tables
CREATE TABLE dim_category (
    category_key    BIGINT        PRIMARY KEY,
    category_code   VARCHAR(20)   UNIQUE NOT NULL,
    category_name   VARCHAR(100)
);

CREATE TABLE dim_brand (
    brand_key       BIGINT        PRIMARY KEY,
    brand_code      VARCHAR(20)   UNIQUE NOT NULL,
    brand_name      VARCHAR(100)
);

CREATE TABLE dim_supplier (
    supplier_key    BIGINT        PRIMARY KEY,
    supplier_name   VARCHAR(200)  UNIQUE NOT NULL
);

-- Step 2: populate sub-dims from distinct values in the star dim
INSERT INTO dim_category (category_key, category_code, category_name)
SELECT ROW_NUMBER() OVER (ORDER BY category_code),
       category_code, category_name
FROM   (SELECT DISTINCT category_code, category_name
        FROM dim_product_star) x;

INSERT INTO dim_brand (brand_key, brand_code, brand_name)
SELECT ROW_NUMBER() OVER (ORDER BY brand_code),

```

```

        brand_code, brand_name
FROM    (SELECT DISTINCT brand_code, brand_name
        FROM dim_product_star) x;

INSERT INTO dim_supplier (supplier_key, supplier_name)
SELECT ROW_NUMBER() OVER (ORDER BY supplier_name),
       supplier_name
FROM    (SELECT DISTINCT supplier_name
        FROM dim_product_star) x;

-- Step 3: create the new snowflaked dim_product with FKs
CREATE TABLE dim_product_snow (
    product_key    BIGINT        PRIMARY KEY,
    sku            VARCHAR(50),
    product_name   VARCHAR(200),
    category_key   BIGINT        REFERENCES dim_category(category_key),
    brand_key      BIGINT        REFERENCES dim_brand(brand_key),
    supplier_key   BIGINT        REFERENCES dim_supplier(supplier_key),
    unit_cost      DECIMAL(12, 2)
);

-- Step 4: migrate data, resolving text values into FKs
INSERT INTO dim_product_snow
SELECT p.product_key, p.sku, p.product_name,
       c.category_key, b.brand_key, s.supplier_key,
       p.unit_cost
FROM    dim_product_star p
JOIN    dim_category      c ON c.category_code = p.category_code
JOIN    dim_brand         b ON b.brand_code   = p.brand_code
JOIN    dim_supplier      s ON s.supplier_name = p.supplier_name;

-- Same query, now 4 joins
-- SELECT c.category_name, SUM(s.net_amount)
-- FROM    fact_sales s
-- JOIN    dim_product_snow p ON p.product_key = s.product_key
-- JOIN    dim_category      c ON c.category_key = p.category_key
-- GROUP BY c.category_name;

-- =====
-- REVERSE: SNOWFLAKE → STAR (denormalize back)
-- =====

CREATE TABLE dim_product_star_v2 AS
SELECT p.product_key, p.sku, p.product_name,
       c.category_code, c.category_name,
       b.brand_code,    b.brand_name,
       s.supplier_name, p.unit_cost

```

```
FROM    dim_product_snow p
JOIN    dim_category      c ON c.category_key = p.category_key
JOIN    dim_brand         b ON b.brand_key    = p.brand_key
JOIN    dim_supplier      s ON s.supplier_key = p.supplier_key;

-- fact_sales.product_key is unchanged in every step above.
-- Only the DIM shape and QUERY shape differ.
```

Student

Can you walk me through **converting a star to a snowflake** - and then back again?

ChatGPT • AI Assistant

This is the ultimate exercise for cementing everything in this chapter. We'll take a **dim_product** from the Chapter 2 star, snowflake it into product + category + brand + supplier sub-dims, then unsnowflake back to a flat dim. You'll see exactly what changes and what stays.

CONCEPT CARD

What the two-way conversion demonstrates:

1. **Star** → **Snowflake** - extract hierarchies out of the dim into new sub-tables. Add FKs. Original dim becomes narrower.
2. **Snowflake** → **Star** - inline all sub-dim attributes back into the main dim. FKs become plain columns.
3. **The fact table is untouched** in both directions - it only knows about **product_key**.
4. **Reports have to change** in both directions - join counts differ.

Student

What's the recipe?

ChatGPT • AI Assistant

EXAMPLE CARD

The star-to-snowflake recipe (applied below):

1. Identify hierarchical attributes in the current dim (category, brand, supplier).
2. Create separate `dim_category` , `dim_brand` , `dim_supplier` tables - each with surrogate PK + attributes.
3. Populate the sub-dims from the distinct values in the current dim.
4. Add FK columns (`category_key` , `brand_key` , `supplier_key`) to the main dim.
5. Drop the original text columns from the main dim.

And the reverse:

1. Bring back the text columns.
2. Populate them via a JOIN + UPDATE from the sub-dims.
3. Drop the FK columns and the sub-dim tables (or keep them for reference).

🔗 ANALOGY

Converting between star and snowflake is like **repacking a suitcase**. Same clothes, different organization. Neatly folded (snowflake) vs stuffed together (star). Both fit the trip; pick based on how often you'll rummage.

⚠️ WARNING

In production, this conversion is done as part of a **schema migration** with careful data validation. Never drop columns before verifying every row of the sub-dim was populated correctly.

🔥 EXERCISE

Copy the DDL below. Run the star version first. Then convert to snowflake. Compare a revenue-by-brand query in both - count the joins. Then reverse. Notice how the fact table doesn't change; only the dim shape and query shape do.

[AI Prompt] Paste this into any AI chatbot: *"Show me the full SQL migration from a star with `dimproduct` (flat, including `category` + `brand` + `supplier`) to a snowflake with `dimproduct` referencing `dimcategory`, `dimbrand`, `dim_supplier`. Include the reverse migration too.*

Explain what breaks if you skip data validation between steps."

CHAPTER REVIEW & SUMMARY

Can you explain what this chapter is about?

ChatGPT • AI Assistant

CHAPTER SUMMARY

This chapter zoomed out from individual tables to overall warehouse **shape**. You now recognize star, snowflake, and galaxy patterns at a glance, and you can defend your choice per dimension with real trade-offs.

- **WHAT IS A SCHEMA IN A DATA WAREHOUSE:** The overall pattern of how facts and dimensions connect - star, snowflake, or galaxy - not the same as OLTP entity-relationship diagrams.
- **THE STAR SCHEMA:** One central fact + fully denormalized dimensions; every attribute is one hop away. Kimball's default because query SQL is short and fast.
- **THE SNOWFLAKE SCHEMA:** A star with dimensions normalized into sub-tables. More tables, more joins, less redundancy - useful for huge dims, independently-governed hierarchies, or compliance-driven reference data.
- **STAR VS SNOWFLAKE - WHEN TO USE WHICH:** Default to star. Choose snowflake for large dims, separately-governed hierarchies, high-frequency hierarchy updates, or reference-data governance. Hybrid ('starflake') is common.
- **GALAXY (FACT CONSTELLATION) SCHEMAS:** Multiple fact tables sharing conformed dimensions - what real enterprise warehouses look like. Enables cross-fact analysis and single-source-of-truth reporting.
- **PERFORMANCE CONSIDERATIONS:** Four levers - FK indexes, date-partitioning, clustering/sort keys, materialized aggregate tables - plus the column-store advantage that makes wide star dimensions cheap.
- **COMMON MODELING PITFALLS:** Mixed grain, missing surrogate keys, fact-to-fact joins, accidental snowflaking, overly wide dims, and descriptive text on facts - six anti-patterns to spot in every code review.
- **HANDS-ON CONVERT STAR TO SNOWFLAKE AND BACK:** The full DDL and data-migration flow for both directions - proving that the fact table stays untouched while the dim shape and query complexity trade off.

Student

Can you break down the next concept?

 ChatGPT • AI Assistant

You've mastered the warehouse-schema patterns. In the next chapter we go up another layer of abstraction: **Data Warehouse vs Data Lake vs Lakehouse** - the architectural choice that determines where and how your star, snowflake, and galaxy schemas actually live. Onward!